

Korrekte Layer-Berechnung für hierarchische Architekturvisualisierungen *Constraints, Invarianten und Pitfalls*

[Entwurf zur Abstimmung — bitte um Rückmeldung bis 17. Mai 2026]

Johannes Weigend

Technische Hochschule Rosenheim / Weigend AM GmbH & Co.KG, Germany
johannes.weigend@gmail.com

Veronika Schwarz

[Affiliation bitte ergänzen]
[E-Mail bitte ergänzen]

Michael Philippsen

Friedrich-Alexander-Universität Erlangen-Nürnberg, Germany
[E-Mail bitte ergänzen]

Keywords: software architecture visualization, layout invariants, layered software city, strongly connected components, dependency analysis, tool demonstration

Abstract: Geschichtete Architekturvisualisierungen drücken Abhängigkeitsstruktur durch vertikale Anordnung aus: wer von anderen abhängt, steht höher. Ein Package auf einer niedrigeren Schicht als seine eigenen Klassen, zwei zyklisch abhängige Pakete auf unterschiedlichen Ebenen, eine reguläre Abhängigkeit die entgegen der Schichtrichtung zeigt — solche Fehler in der Layer-Berechnung sehen plausibel aus, täuschen aber den Betrachter. Das Prinzip klingt selbstverständlich: die Korrektheit eines Architekturlayouts sollte das Werkzeug garantieren, nicht der Nutzer. Wie schwer das tatsächlich ist, zeigt erst die Implementierung.

Unser 2021 bei IVAPP ausgezeichnetes Paper (Dashuber et al., 2021) führte ein Schichtlayout ein, dessen Grundprinzip einfach ist: jede Klasse erhält ein höheres Level als die Klassen von denen sie abhängt. Die X/Y-Position von Gebäuden auf dem Stadtgrundriss folgt dieser Ordnung direkt — foundationale Klassen stehen vorne/unten, hochrangige Module hinten/oben. Gebäudehöhen kodieren orthogonal dazu Metriken wie Komplexität oder Zeilenzahl. Dieser Beitrag stellt eine quelloffene Referenzimplementierung (Java, Apache 2.0) vor sowie einen **Layout-Invarianten-Checker** der vier formal spezifizierte Nachbedingungen nach jedem Analyselauf verifiziert.

Die Pipeline selbst ist nicht trivial. Ein korrektes Schichtlayout erfordert die Lösung dreier interdependenter Constraint-Systeme — Klassen-Ordnung, Package-Hierarchie und SCC-Gleichheit. Ein naiver Single-Pass-Algorithmus liefert bei zyklischen Graphen nicht-deterministische Ergebnisse. Der konzeptionelle Kern der Lösung: die Pipeline berechnet **zwei semantisch verschiedene Level-Konzepte**. Phase 1 berechnet die *architektonische Tiefe* als längsten Pfad im globalen, zyklensfreien SCC-DAG. Phase 2 berechnet die *lokale Layout-Position* innerhalb eines Packages, beginnend bei 0, ausschließlich auf Geschwister-Abhängigkeiten. Stark zyklischer Code — Minecraft Forge: eine SCC aus 4.038 Klassen mit 3.585 Rückkanten — wird durch einen heuristischen SCC-Breaker (Eades et al., 1993) zu einer nutzbaren Hierarchie aufgelöst.

Der Invarianten-Checker (R1–R5) unterscheidet präzise zwischen Algorithmusfehlern und tolerierten Heuristik-Verletzungen. Die Entwicklung dieser Regeln war selbst nicht fehlerfrei: eine frühe Variante von R2 erwies sich als zu grob und musste verfeinert werden. Der Checker validiert damit nicht nur die Pipeline, sondern auch die Präzision der Regeln selbst — ein doppelter Boden den kein manueller Review ersetzen kann. Dieselben vier Regeln wurden portiert auf eine unabhängige 3D-Software-City in Unity/.NET — mit byte-identischen Befunden auf gemeinsamen Eingaben. Ein R2-Befund in der 92-Modul-Codebasis *software-ekg-7* identifizierte einen fehlenden Package-SCC-Equalisierungsschritt und löste direkt einen Pipeline-Fix aus. Nach unserem Kenntnisstand hat keine frühere geschichtete Architekturvisualisierung diesen Grad an Selbstvalidierung realisiert.

REFERENCES

- Dashuber, V., Philippsen, M., and Weigend, J. (2021). A layered software city for dependency visualization. In *Proceedings of the 12th International Conference on Information Visualization Theory and Applications (IVAPP)*, VISIGRAPP 2021, pages 277–285. SCITEPRESS. Best Paper Award.
- Eades, P., Lin, X., and Smyth, W. F. (1993). A fast and effective heuristic for the feedback arc set problem. *Information Processing Letters*, 47(6):319–323.

ANHANG A — Warum Single-Pass nicht funktioniert: eine Analyse der Fehlermodi

Das Grundproblem: drei interagierende Constraint-Systeme

Ein korrektes Schichtlayout muss gleichzeitig drei Constraints erfüllen:

1. **Klassen-Constraint:** Wenn $A \rightarrow B$ (A hängt von B ab), dann muss $\text{level}(A) > \text{level}(B)$.
2. **Hierarchie-Constraint (R3):** Ein Package muss auf mindestens dem Level seines höchsten Inhalts liegen.
3. **SCC-Constraint (R2):** Alle Klassen in einer zyklischen Abhängigkeit bekommen dasselbe Level; alle Packages die eine Package-SCC bilden ebenso.

Diese drei Constraints sind einzeln lösbar. Zusammen erzeugen sie ein **zirkuläres Abhängigkeitssystem zwischen den Ebenen der Berechnung selbst** — nicht zwischen den Klassen.

Die konkreten Fehlermodi eines Single-Pass-Ansatzes

Fehlermodus 1: Retroaktive Korrektur. Der Algorithmus iteriert über die Klassen und vergibt Level. Er sieht Klasse A (in Package P_1), berechnet $\text{level}(A) = 3$, setzt damit $\text{level}(P_1) = 3$. Weiter trifft er Klasse B (in P_2) und stellt fest: A und B sind in einer SCC, müssen gleiches Level bekommen, jetzt $\text{level}(A) = \text{level}(B) = 5$. Damit muss $\text{level}(P_1)$ neu berechnet werden — aber P_1 und P_3 wurden bereits abgeschlossen. Die SCC-Mitgliedschaft ist erst nach vollständiger Graphtraversierung bekannt.

Fehlermodus 2: Verwobene Zykluserkennung erzeugt classpath-abhängige Ergebnisse. Klassen kommen aus JARs. Die Reihenfolge folgt der JAR-Packreihenfolge, die abhängt von Build-Tool (Maven vs. Gradle), Modul-Reihenfolge und Dateisystem-Sortierung — keine davon codiert Abhängigkeitsrichtung. Derselbe Quellcode liefert so unterschiedliche Level-Layouts je nach Build-Konfiguration. Die Fehler sind **subtil**: Das Layout sieht plausibel aus, Pfeile zeigen grob in die richtige Richtung, die präzise Hierarchieinformation ist korrumpiert aber nicht sofort sichtbar.

Fehlermodus 3: Gegenseitiger Verweis zwischen Package- und Klassen-Level. $\text{level}(P_1) = \max(\text{level}(c) \mid c \in \text{subtree}(P_1))$. Welches Level hat die höchste Klasse? Das hängt von anderen Klassen ab, die in Packages liegen, deren Level davon abhängt. . . Dieser zirkuläre Verweis macht einen gemeinsamen Pass über Klassen und Packages inhärent instabil.

Fehlermodus 4: Der große See. Ohne Mechanismus zum Aufbrechen großer SCCs landen alle Mitglieder zwangsläufig auf demselben Level. Bei Minecraft Forge — eine SCC aus 4.038 Klassen — liegen über 40 % aller Klassen auf Level 0: ein riesiger, flacher See am unteren Rand des Layouts. Der heuristische SCC-Breaker (In-/Out-Degree-Rankscore, verwandt mit (Eades et al., 1993)) löst dieses Problem. Entfernte Kanten werden als Verletzungen (rot) visualisiert, nicht ignoriert.

Formaler Beweis: Warum ein naiver rekursiver Algorithmus versagt

Der intuitivste Ansatz ist ein rekursiver DFS mit Memoization:

```
level(A) :
  if A already computed: return level(A)
  if A has no dependencies: return 0
  return max(level(dep) for dep in A.dependencies) + 1
```

Für DAGs korrekt. Für zyklische Graphen versagt er strukturell.

Gegenbeispiel (zwei verschachtelte SCCs): $\text{SCC}_1 = \{A, B\}$ mit $A \rightarrow B \rightarrow A$, $\text{SCC}_2 = \{C, D\}$ mit $C \rightarrow D \rightarrow C$, Querbängigkeit $A \rightarrow C$. Korrekte Level: $\text{level}(C) = \text{level}(D) = 0$, $\text{level}(A) = \text{level}(B) = 1$.

Rekursiver Algorithmus, startend bei A :

```
level(A) [Sentinel = 0]
-> level(B) -> level(A): Zyklus, return 0 => B.level = 1
-> level(C) [Sentinel = 0]
  -> level(D) -> level(C): Zyklus, return 0 => D.level = 1
```

```
C.level = 2    <- FALSCH (korrekt waere 0)
A.level = max(1,2)+1 = 3    <- FALSCH (korrekt waere 1)
```

Satz: Ein rekursiver Level-Algorithmus mit Sentinel-basierter Zykluserkennung produziert für Graphen mit mehreren interagierenden SCCs Ergebnisse, die von der Traversierungsreihenfolge abhängen. Da die Traversierungsreihenfolge bei JVM-basierter Klassenladung der Classpath-Reihenfolge folgt, sind die Ergebnisse **nicht deterministisch über verschiedene Build-Konfigurationen**.

Die Lösung: zwei semantisch verschiedene Level-Konzepte

Die Fehlermodi 1–3 haben eine gemeinsame Ursache: der Versuch, *architektonische Tiefe* und *lokale Layoutposition* in einem einzigen Level-Begriff zu vereinen. Die Pipeline trennt beide Konzepte konsequent:

- **Phase 1** berechnet die *architektonische Tiefe* — den längsten Pfad im globalen, zyklensfreien SCC-DAG über **alle Klassen des Projekts**.
- **Phase 2** berechnet die *lokale Layout-Position* innerhalb eines Packages, beginnend bei 0, ausschließlich auf Geschwister-Abhängigkeiten.

```
Phase 1: Klassen-Graph -> Tarjan -> SCC-DAG (azyklisch)
        -> Longest-Path -> globale Level
        -> SCC-Equalisierung (alle Mitglieder = Maximum)
        -> Package-Level (Propagation nach oben)
        [Iterationsloop: Step 5 + Step 4b bis stabil]
```

```
Phase 2: Pro Package: Geschwister-Graph (nur interne Kanten)
        -> Tarjan -> lokaler SCC-DAG
        -> Longest-Path -> lokale Visualisierungs-Positionen
```

Hinweis: Nach Phase 1 ist Level keine Eigenschaft einer einzelnen Klasse, sondern des SCC-Knotens dem sie angehört — gemessen als längster Pfad im zyklensfreien Quotienten-Graphen (SCC-DAG) über alle Klassen des Projekts.

	Phase 1	Phase 2
Bedeutung	Architekton. Tiefe im SCC-DAG	Layout-Position unter Geschwistern
Scope	Global — alle Klassen	Lokal — Kinder eines Packages
Startpunkt	Blattklassen = 0	Immer 0 für unterstes Element
Zyklen	SCC-Kollaps vor Berechnung	Lokaler Tarjan
Wertebereich	Hunderte möglich	Geschwisteranzahl – 1

Table 1: Vergleich der beiden Level-Konzepte

R4: eine verworfene Regel — und was das lehrt

Eine frühe Fassung enthielt R4: „*Zyklisch voneinander abhängige Kind-Packages müssen dasselbe Level teilen.*“ Die Regel scheiterte in der Praxis: Parent↔Child-Kanten, Rückkanten des SCCBreakers und gemeinsame Klassen-SCCs erzeugten Schein-Zyklen und fälschliche Equalisierungen. R4 wurde durch R2 ersetzt, das denselben Anspruch mit dem richtigen gefilterten Graphen umsetzt.

Die eigentliche Lehre: Constraints für ein komplexes System korrekt zu formulieren ist selbst schwer. Der Checker schafft einen *doppelten Boden*: er validiert nicht nur die Pipeline-Implementierung gegen die Regeln, sondern deckt auch auf wenn die Regeln selbst zu grob sind. Dass R2 danach einen echten Pipeline-Bug in *software-ekg-7* fand, beweist dass die verschärfte Regel präzise genug war um echte Fehler von Schein-Fehlern zu trennen.